

Components & Props

Build Reusable UI with Composition

■ LEARNING OUTCOME

By the end of this module you will design component hierarchies, pass data with props, use TypeScript for type safety, apply composition patterns, and build reusable components.

■ Skills You Will Gain

- Design component hierarchies from a UI mockup
- Pass and receive props including function callbacks
- Use TypeScript interfaces for prop types
- Apply children and composition patterns
- Build compound components
- Create reusable UI component library

■ Components are the core abstraction of React. Every UI element is a component. Mastering components and props is mastering React.

SECTION 1

Props Fundamentals

```
// Props are READ-ONLY -- never mutate props! function App() { const user = { id:1, name:'Alice',
role:'admin' }; const handleDelete = (id) => console.log('Delete:', id); return ( <div> <Greeting
name='Alice' age={28} isLoggedIn={true} /> <UserCard user={user} /> <DeleteButton userId={user.id}
onDelete={handleDelete} /> <Card title='Report'><p>Revenue: Rs.1,20,000</p></Card> </div> ); } //
Receive props with destructuring function Greeting({ name, age, isLoggedIn = false }) { return
<h1>{isLoggedIn ? `Hi ${name} (${age})!` : 'Please log in'}</h1>; } // Nested destructuring
function UserCard({ user: { name, role } }) { return <div className={`card
role-${role}`}>{name}</div>; } // Callback prop -- child calls it to inform parent function
DeleteButton({ userId, onDelete }) { return <button onClick={() =>
onDelete(userId)}>Delete</button>; } // children prop function Card({ title, children }) { return (
<div className='card'> <div className='card-header'><h3>{title}</h3></div> <div
className='card-body'>{children}</div> </div> ); }
```

SECTION 2

TypeScript Props and Composition

```
// TypeScript interface interface ButtonProps { label: string; variant?: 'primary' | 'secondary' |
'danger'; size?: 'sm' | 'md' | 'lg'; disabled?: boolean; loading?: boolean; onClick?: () => void;
children?: React.ReactNode; } const Button: React.FC<ButtonProps> = ({ label, variant='primary',
size='md', disabled=false, loading=false, onClick, children, }) => ( <button className={`btn
btn-${variant} btn-${size}`} disabled={disabled || loading} onClick={onClick} > {loading && <span
className='spinner' aria-hidden='true' />} {children || label} </button> ); // Higher-Order
Component function withAuth(WrappedComponent) { return function AuthenticatedComponent(props) {
const { user, isLoading } = useAuth(); if (isLoading) return <LoadingSpinner />; if (!user) return
<Navigate to='/login' />; return <WrappedComponent {...props} user={user} />; }; } const
ProtectedDashboard = withAuth(Dashboard); // Compound components function Select({ children, value,
onChange }) { return ( <SelectContext.Provider value={{ value, onChange }}> <div
className='select'>{children}</div> </SelectContext.Provider> ); } Select.Option = function({
value, children }) { const { value:sel, onChange } = useContext(SelectContext); return <div
className={sel===value?'selected':''} onClick={()=>onChange(value)}>{children}</div>; }; // Usage:
// <Select value={v} onChange={set}> // <Select.Option value='a'>Option A</Select.Option> //
</Select>
```

■ PRACTICE Q & A

Q1. What is prop drilling and how do you avoid it?

A: Passing props through many intermediate components. Avoid with: React Context, state management library (Zustand), or component composition (move consumer closer to data).

Q2. Why are props read-only?

A: One-directional data flow keeps state predictable. To send data UP, pass a callback function as a prop -- child calls it, parent handles state change.

Q3. What is the children prop?

A: Special prop containing JSX between opening and closing tags. Can be text, elements, arrays, or functions (render props).

Q4. What is a controlled component?

A: Form element value controlled by React state (value={state} onChange={setState}). Uncontrolled uses DOM state (useRef). Prefer controlled for validation.

Q5. What does spreading props do?

A: Passes all properties of an object as individual props. Useful for forwarding props but use carefully to avoid passing unwanted props to DOM elements.

■ Components & Props Cheat Sheet	
<code>function Comp({ a, b='default' })</code>	Destructure with defaults
<code>{ user: { name, role } }</code>	Nested destructuring
<code>children prop</code>	JSX between opening/closing tags
<code>{...props}</code>	Spread all props to child
<code>React.FC<Props></code>	TypeScript functional component
<code>interface Props { name: string }</code>	TypeScript prop types
<code>variant?: 'a' 'b'</code>	Optional union type prop
<code>React.ReactNode</code>	Any valid JSX as prop type
<code>withAuth(Component)</code>	Higher-Order Component
<code>Comp.Sub = SubComp</code>	Compound component pattern
<code>Context.Provider value={...}</code>	Deep prop sharing
<code>{...rest}</code>	Forward remaining props